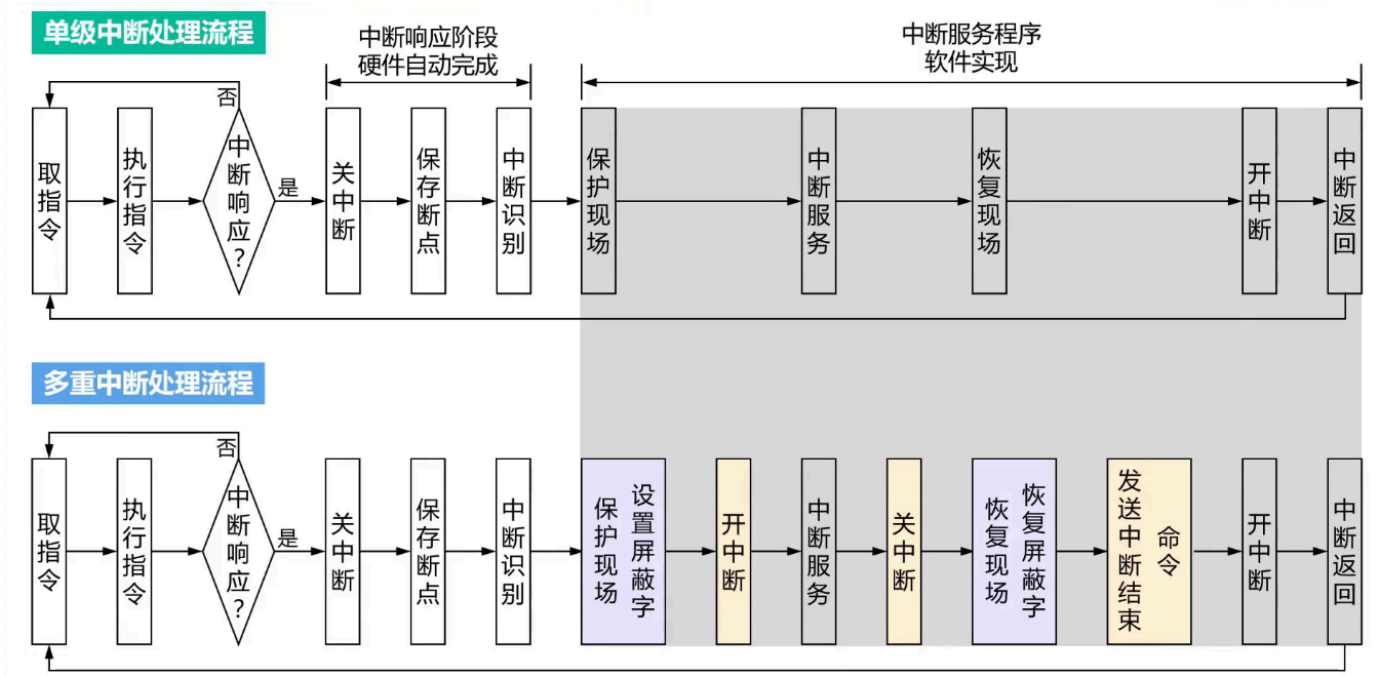
一、流程图



这张对比图的精髓就一句话：**多重中断 = 单级中断的骨架 + 屏蔽字机制 + 两次开关中断。**上下两图灰色重叠区是共同的（中断响应三连硬件完成），下半图新出现的三类框（紫=屏蔽字、黄=开关中断与EOI）才是考点所在

与单级中断的三个差异点

新增步骤	位置	作用
设置屏蔽字	保护现场之后、开中断之前	定制"允许谁打断我"，开中断前必须先设好
第一次开中断	设置屏蔽字之后、中断服务之前	放行嵌套：让更高优先级请求能进来
第二次关中断	中断服务之后、恢复现场之前	锁门护场：保证恢复现场不被打断
发送中断结束命令（EOI）	恢复现场之后	通知8259A清ISR位，放行后续同级/低级请求
### 关于几次中断的开关		

- **第一次开中断**：只放行**处理优先级更高**的请求（低级和同级被屏蔽字挡住，仍登记在8259A的IRR里等着）；
- **第二次关中断**：保护"恢复现场+恢复屏蔽字"这段操作的原子性——现场恢复到一半被改写就全乱了；
- **最后开中断**：返回主程序前恢复CPU的响应能力。注意此时**断点PSW的弹出（IRET）会连同IF位一起恢复**，这里体现为流程图上的显式开中断

EOI命令的位置（呼应8259A那张图）

"发送中断结束命令"就是CPU向8259A发**EOI**，清ISR对应位。放这里的逻辑：现场和屏蔽字都恢复了，才轮到告诉8259A"这个中断我彻底办完了"。如果漏发，ISR位一直是1，同级和低级中断从此进不来（PR判优时压不过正在服务的那个）

易错点清单

- **易错1：认为开中断后同级/低级也能嵌套进来。** 错。能插进来的只有**处理优先级更高**的；同级和低级被屏蔽字挡住，只是继续在IRR里挂起，不会丢失

- **易错2：混淆两种优先级。** “同时到达谁先被响应”看**响应优先级**（硬件排队器）；“正在执行时谁能打断我”看**处理优先级**（屏蔽字）。题目给两个次序让你填屏蔽字，考的全是处理优先级
- **易错3：屏蔽字设置与开中断的顺序颠倒。** 必须先设屏蔽字再开中断；恢复时先恢复现场和屏蔽字再开中断。任何“开中断裸奔在防护未设好时”的选项都是错的
- **易错4：认为屏蔽字恢复可以省略。** 屏蔽字若不恢复，下一层嵌套返回后，本层设的屏蔽还在生效，会错误地持续屏蔽其他中断源
- **易错5：把“设置屏蔽字”当成隐指令。** 屏蔽字是**服务程序里用特权指令设置的**（软件），不是硬件自动完成的。隐指令三件事（关中断/存断点/中断识别）在上下两图中**完全相同**，灰色区重叠正是这个含义
- **易错6：认为多重中断每一层的流程不一样。** 每一层嵌套进来的ISR都执行同一套模板（设屏蔽字→开→服务→关→恢复→EOI→开→返回），嵌套只是把这套模板叠罗汉
- **易错7：漏掉“中断请求不丢失”的理解。** 被屏蔽的请求不会消失，它在8259A的IRR里保持挂起，等屏蔽解除（或ISR清零）后PR重新判优——这也解释了为什么低级中断只是“延迟处理”而非“丢弃”。
  一句话收束：**多重中断的流程变化全在软件段——进场先设屏蔽字再开门，出门先锁门再恢复、补发EOI再走人；屏蔽字让“谁打断谁”从硬件铁律变成软件可调，这就是响应优先级与处理优先级的分野**

二、多重中断服务程序与子函数的对比

下表展示了中断服务程序（一种由事件驱动的、由硬件密切管理的特殊程序）与子函数（一种由软件控制的、逻辑复用的常规程序）在核心机制上的本质区别。

对比维度	中断服务程序	子函数
调用方式	由硬件或软件中断事件随机触发，无明确调用者。	在程序中通过调用指令显示调用，调用时机由代码逻辑决定。
PC的保存与修改	● 由硬件中断隐指令自动完成。 ● 入口地址由中断识别机制（如中断向量表）提供。	● 由专用调用指令（如x86的call、MIPS的jal）完成。 ● 入口地址由调用指令本身编码给出。
保护现场的内容	● 更多的寄存器。 ● 由于没有“调用者”，除了自身要修改的寄存器，还需额外保存ABI（Application Binary Interface）中约定的由“调用者保存的寄存器”，也就是所有会被中断服务程序改写的寄存器。	● 部分寄存器。 ● 只需按约定保存自身会修改的“被调用者保存寄存器”。
返回主程序的方式	● 中断服务程序返回与函数调用返回使用的指令不同。 ➢ 对于中断服务程序的返回，x86使用iret，MIPS使用eret。 ➢ 对于函数调用的返回，x86使用ret，MIPS使用jr $ra。	